# SWC3

## Software-Entwicklung mit C und C++

Erik Pitzer

WS 2026/27

FH OÖ - Campus Hagenberg - Software Engineering



# 14. C++ Fehlerbehandlung & Standardbibliothek

# 14.1 C++ Fehlerbehandlung

# Fehlerbehandlung in C++

- verschiedene Möglichkeiten Fehler zu behandeln
  - Rückgabewert
    - einfach zu implementieren
    - eher unsicher, Rückgabewert kann oft leicht ignoriert werden
  - globale Variable (z.B. C `ERRNO`)
    - einfach für Auslöser
    - umständlich in Behandlung
  - Programmabbruch (`std::terminate()` → `std::abort()`)
    - einfach aber oft zu radikal
  - Ausnahmen
    - einfach für Auslöser
    - einfach für Behandlung
    - aufwändiger für Compiler und zur Laufzeit

# Fehlerbehandlung ohne Ausnahmen

- Fehlerbehandlung oft nicht direkt möglich
- Fehler muss propagiert werden → kann leicht übersehen werden

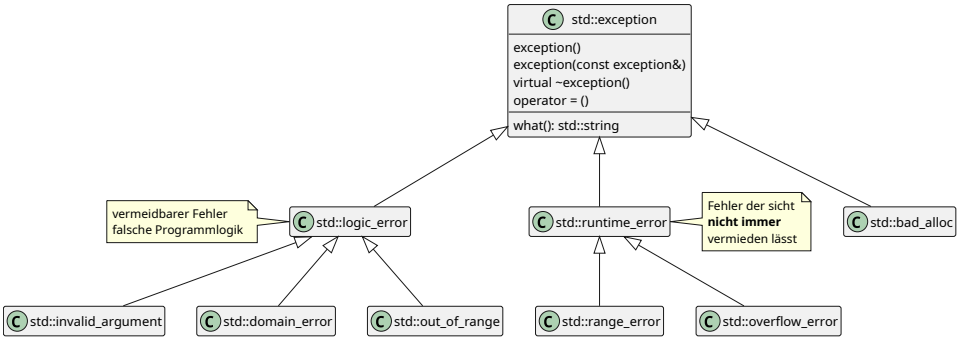
```
URL* open(std::string url) {
  // sets ERRNO on error
}

int scrape(std::string url) {
  // ...
  URL* u = uopen(url); // XXX: might discard error
  // return -1 on error
}

int main() {
  std::string url = ask_user();
  scrape(url); // XXX: discard error
  // ...
  // XXX: apparent error often later
}
```

# Ausnahmen in C++ (*exceptions*)

- Auslösen mit Schlüsselwort »`throw`«
- Behandeln (“*fangen*”) mit Schlüsselwort »`catch`« am Ende eines »`try`«-Blocks
  - unbehandelte Ausnahmen führen zum Programmabbruch über »`std::terminate()`«
- typischerweise wird per Value *geworfen* und per Referenz *gefangen*
  - also `throw std::exception()` und nicht »`throw new std::exception()`«
- einige vordefinierte Exceptions in der Standardbibliothek
  - Fehlermeldung in Methode `std::exception::what()`



# Ausnahmen in C++: Beispiele

- Weiterleitung ohne spezielle Vorkehrungen

```
void exceptional() {
  throw std::exception(); // throw value
}

void boring() { // pass exceptions
  exceptional(); // no special handling
}

int main() {
  try {
    boring();
  } catch (std::exception &x) { // catch reference
    std::println("{}", x.what());
  }
}
```

```
std::exception
```

- Reihenfolge der `catch`-Statements
  - zuerst spezifischer → dann generischer

```
template<typename X, typename... Args>
void test(Args&&... args) {
  try {
    throw X{args...};
  } catch (std::logic_error &x) { // specific
    PL("std::logic_error: {}", x.what());
  } catch (std::exception &x) { // more general
    PL("std::exception: {}", x.what());
  } catch (std::string &message) {
    PL("std::string: {}", message);
  } catch ( ... ) { // catch all
    PL("something caught");
    // throw; // NOTE: re-throw
  }
}
int main() {
  test<std::overflow_error>("too big");
  test<std::out_of_range>("outside");
  test<std::string>("Hi there!");
  test<int>();
}
```

```
std::exception: too big
std::logic_error: outside
std::string: Hi there!
something caught
```

# noexcept & std::terminate

- Funktionen und Methoden können mit »`noexcept`« markiert werden
  - drückt aus dass hier keine Ausnahme passieren kann
  - erlaubt Optimierung durch Compiler
- bei unerwarteten oder unbehandelten Ausnahmen → `std::terminate()`

```
void safe() noexcept {
  throw std::exception(); // XXX: throw exception despite noexcept -> unexpected
}

void terminate_handler() {
  std::println("terminate: emergency stop");
  exit(0); // otherwise exit(1);
}

int main() {
  std::set_terminate(terminate_handler);
  try {
    safe();
    std::println("done?");
  } catch (...) { // XXX: exception not caught
    std::println("saved?");
  }
}
```

```
terminate: emergency stop
```

# Vorschau: `std::optional` (C++17) && `std::expected` (C++23)

- moderne, sichere und performante Alternative, explizite Markierung von Wert und/oder Fehler
  - Schnittstelle beschreibt alle Möglichkeiten, verpackt Ergebnis (oder Fehler) → explizite Unterscheidung

```cpp
std::optional<double> safe_sqrt(double d) {
  if (d < 0) return std::nullopt;
  return sqrt(d);
}

std::expected<double, std::domain_error> safe_div(double x, double d) {
  if (d == 0) return std::unexpected(std::domain_error("division by zero"));
  return x / d;
}

int main() {
  SHOW(safe_sqrt(2).value_or(0));
  SHOW(safe_sqrt(-2).has_value());
  SHOW(safe_div(1.0, 2.0).value());
  SHOW(safe_div(1.0, 0.0).value_or(0.0));
}
```

```
safe_sqrt(2).value_or(0) = 1.4142135623730951
safe_sqrt(-2).has_value() = false
safe_div(1.0, 2.0).value() = 0.5
safe_div(1.0, 0.0).value_or(0.0) = 0
```

# 14.2 C++ Standardbibliothek

# Überblick C++-Standard-Bibliothek

- sehr umfangreich
  - besonders im Vergleich zur C-Standardbibliothek
- C-Header mit minimalen Änderungen verfügbar
  - z.B. <cstdlib>, <climits>, <cstdint>, <cmath>
- Zeichenketten <string>
- Reguläre Ausdrücke, <regex>
- Ein-/und Ausgabe <iostream>, <fstream>, ...
- Umgang mit Dateisystem und Pfaden <filesystem>
- Container <vector>, <set>, <map>, <queue>, <deque>, <stack>, ...
  - darauf operierenden Algorithmen <algorithm>, <iterator>
- Zufallszahlen und -verteilungen <random>
- Zeit- und Datumstypen und -funktionen <chrono>
- Runtime-Type-Information <typeinfo> ermöglicht (sehr begrenzte) Information über dynamischen Datentyp
- Parallele Datenverarbeitung <atomic>, <future>, <mutex>, <thread>, ...

Standardbibliothek möglichst oft nutzen und nicht *das Rad neu erfinden*

C++ Standard Library headers - cppreference.com

# Zeichenketten

- Zeichenketten (std::string) sind vergleichbar mit Vektor aus Buchstaben (`std::vector<char>`), bieten aber zusätzliche Funktionalität

```
std::string s { "one" };
s += " two";
s.append(" three");
s.push_back('!'); // NOTE: single character
SHOW(s);

SHOW(s.substr(4, 3));
s.replace(4, 3, "twelve");
SHOW(s);

s[9] = '!'; // array-like access
// s[20] = '!'; XXX: undefined behaviour
// s.at(20) = '!'; -> std::out_of_range exception
SHOW(s);

printf("%s", s.c_str());
```

- Beispiele
  - Teilzeichenkette extrahieren (`substr()`)
  - oder ersetzen (`replace()`)
  - in C-Zeichenkette umwandeln (`c_str()`)

```
s = "one two three!"
s.substr(4, 3) = "two"
s = "one twelve three!"
s = "one twelv! three!"
one twelv! three!
```

Anwendung immer dort wo Zeichenketten auch verändert werden sollen

# String-Views

- `std::string_view` als leichtgewichtige lesende Sicht auf beliebige Zeichenketten
  - z.B. `const char[]`, oder `const std::string&`

```
std::string join(std::string_view s1, std::string_view s2) {
  s1.remove_prefix(1);
  s2.remove_suffix(1);
  std::string result { s1 };
  result += " ";
  result += s2;
  return result;
}

int main() {
  std::string w { "world" };
  SHOW(join("hello", w)); // compatible with both `const char[]` and `std::string` argument
}
```

```
join("hello", w) = "ello worl"
```

- bevorzugt verwenden, wenn nur lesender Zugriff benötigt wird
  - da auch mit `const char[]` und `const char*` kompatibel

Achtung: String-View verlängert nicht die Lebenszeit der zugrundeliegenden Zeichenkette

std::basic_string_view - cppreference.com

# Zeichenketten-Literale

- Zeichenketten-Literale in C++ sind (wie in C) vom Typ `const char[]`
  - C++ bietet Operatoren für kompakte Umwandlung
    - `operator ""s` → `std::string`
    - `operator ""sv` → `std::string_view`

```
SHOW("c-str");

using namespace std::string_literals;
SHOW("std::string"s);
//   ~~~~~~~~~~~~~^  string literal suffix

using namespace std::string_view_literals;
SHOW("std::string_view"sv);
//   ~~~~~~~~~~~~~~~~~~^^   string_view literal suffix
```

```
"c-str" = "c-str"
"std::string"s = "std::string"
"std::string_view"sv = "std::string_view"
```

# Exkurs: Eigene Literale

- durch Überschreiben spezieller Operator-Funktionen lassen sich eigene Literale definieren
  - diese müssen mit einem Unterstrich beginnen, z.B. `operator ""_c ()` für `_c`
- mögliche Quelltypen sind Zeichenketten- oder Zahlen-Literale
  - z.B. `const char*`, `unsigned long int`, `long double`

```
struct Complex {
  long double re {}, im {};
  Complex(long double re = 0.0, long double im =
  ↪  0.0)
    : re{re}, im{im} {}
};

Complex operator ""_i(long double nr) {
  return {0, nr};
}
```

```
int main() {
  auto c = 13.0_i;
  SHOW(c.re, c.im);
  auto i = 1.0_i;
  SHOW(i.re, i.im);
}
```

```
c.re = 0; c.im = 13
i.re = 0; i.im = 1
```

User-defined literals (since C++11) - cppreference.com

# Reguläre Ausdrücke

- Reguläre Ausdrücke sind seit C++11 Bestandteil der Standardbibliothek (Header <regex>)
  - `std::regex` regulärer Ausdruck
  - `std::match_results` Ergebnis
    - `std::sub_match` übereinstimmende Zeichenkette
  - `std::regex_search` Suche nach Muster
  - `std::regex_match` auf Übereinstimmung prüfen

```
std::regex hex_nr_regex { "0x([0-9a-f]+)" };
std::string str { "canyouspotthe0x123abchexnumber?" };
std::smatch string_match;

std::regex_search(str, string_match, hex_nr_regex);

SHOW(string_match[0], string_match[1]);
SHOW(string_match.prefix(), string_match.suffix());
```

Regular expressions library (since C++11) - cppreference.com

# String-Streams

- wie Dateien, können Zeichenketten als Ströme verarbeitet werden (<sstream>)
  - ermöglicht Wiederverwendung von Ein- und Ausgabeoperatoren (`operator >>` und `operator <<`) und Format-Funktionen (`std::format()` und `std::print()`)

```
std::ostringstream numbers;
numbers << 123 << ", " << 456; // formatted output
std::print(numbers, " ({}, {}, {})", 1, 2, 3);
SHOW(numbers.str());

std::istringstream input { "123 abc 123" };
int i, j;
std::string word;
input >> i >> word >> j; // parse stream into numbers
SHOW(i, word, j);
```

```
numbers.str() = "123, 456 (1, 2, 3)"
i = 123; word = "abc"; j = 123
```

# Behälterklassen in C++

- Behälter-Bibliothek enthält *generische* Behälterklassen
- im Unterschied zu Java
  - keine Typ-Hierarchie, keine Interfaces
  - stattdessen lose Kopplung über verschiedene Iterator-Klassen
- Sequenz-basierte Behälter
  - `std::array<T, Size>` fortlaufender Behälter mit fixer Elementanzahl
  - `std::vector<T>` fortlaufender Behälter mit variabler Elementanzahl
  - `std::deqeue<T>` (*double-ended queue*) nicht-fortlaufender Behälter mit Indexzugriff und konstanten Einfüge- & Löschoperatoren an beiden Enden
  - `std::forward_list<T>` lineare List
  - `std::list<T>` doppelt verkettete lineare Liste
- sortierte assoziative Behälter (schnelle Suche)
  - `std::set<T>` mathematische Menge von eindeutigen Elementen
  - `std::map<K, V>` Sammlung von Schlüssel-Wert Paaren mit eindeutigem Schlüssel
  - `std::multiset<T>` & `std::multimap<K, V>` erlauben mehrfaches Vorkommen von Schlüsseln
- unsortierte assoziative Behälter (Hash-basiert)
  - `std::unordered_set<T>`, `std::unordered_map<K, V>`
  - `std::unordered_multiset<T>`, `std::unordered_multimap<K, V>`

Containers library - cppreference.com

# Laufzeitverhalten

| Container | `push_front()` | `insert()` | `push_back()` | `[]` / `at()` |
|---|---|---|---|---|
| `array` | - | - | - | O(1) |
| `vector` | O(n) | O(n) | O(1) | O(1) |
| `list` | O(1) | O(1) | O(n) | - |
| `deque` | O(1) | O(n) | O(1) | O(1) |
| `map` / `set` | - | O(log n) | - | O(log n) |
| `unordered map` / `set` | - | O(1) | - | O(1) |

- darüber Hinaus gibt es noch, z.B.
  - `emplace_back()` statt `push_back()` das ein Objekt direkt im Container anlegt anstatt es dort hinein zu kopieren
  - `data()` um Zugriff auf das zugrundeliegende Feld zu erhalten

# Behälter Initialisierung

Verschiedene Arten der Initialisierung

```
template<typename T = int>
struct vector {
  vector() { PL("vector(default)"); }
  vector(int capacity) {
    PL("vector(capacity = {})", capacity);
  }
  vector(std::initializer_list<T> list) {
    PL("vector(list = {})", list);
  }
};
```

- Default-Konstruktor mit »{}«

```
vector v1;   // -> default
vector v2{}; // -> default
vector v3 = {}; // -> default (copy elision)
```

- Initialisierungsliste, ebenfalls mit »{}«

```
vector v1{1}; // -> elements
vector v2 = {1}; // -> elements (copy elision)
```

- Vorsicht bei »()« → andere Bedeutung

```
vector v(1); // -> NOTE: capacity
```

- andere Varianten mit »()« vermeiden

```
vector v1 = 1; // XXX: implicit constructor
vector v2 = (1); // XXX: implicit constructor
// vector v3(); // XXX: function declaration
// vector v4 = (); /// XXX: invalid syntax
```

# Paare und Tupel

- einfache Datenkapseln mit verschiedenen Datentypen
  - Elementzugriff mit `std::get<I>`-Funktion
  - ermöglicht strukturierte Zuweisung (*structured binding*)

```
int main() {
  std::pair p = std::make_pair(1.1, 'a');
  SHOW(p.first, p.second);
  SHOW(get<0>(p), get<1>(p));

  std::tuple t = std::make_tuple(1, 'a', "Hello");
  SHOW(get<0>(t), get<1>(t), get<2>(t));

  auto [a, b, c] = t; // structured binding (constructor)
  SHOW(a, b, c);

  std::tie(a, b, c) = t; // structured binding (assignment)
  SHOW(a, b, c);
}
```

```
p.first = 1.1; p.second = 'a'
get<0>(p) = 1.1; get<1>(p) = 'a'
get<0>(t) = 1; get<1>(t) = 'a'; get<2>(t) = "Hello"
a = 1; b = 'a'; c = "Hello"
a = 1; b = 'a'; c = "Hello"
```

# Structured Binding für eigene Typen

- entweder durch Ableiten vom entsprechenden Tuple-Typ
- oder durch Überladen spezieller Klassen und Funktionen

```
struct Complex : std::tuple<double, double> {
  constexpr double& re() { return get<0>(*this); }
  constexpr double& im() { return get<1>(*this); }
  template<typename... Args>
  Complex(Args&&... args)
    : std::tuple<double, double>{
        std::forward<Args>(args)...} {
  }
};

int main() {
  Complex c {2, 3};
  double re, im;
  // double &[re, im] = c; XXX
  std::tie(re, im) = c;
  SHOW(c.re(), c.im(), re, im);
}
```

```
c.re() = 2; c.im() = 3; re = 2; im = 3
```

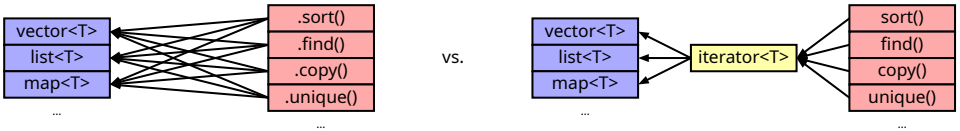
```
struct Complex { double re{}, im{}; };

template <> struct std::tuple_size<Complex> {
  static constexpr size_t value = 2;
};
template <> struct std::tuple_element<0, Complex> {
  using type = double;
};
template <> struct std::tuple_element<1, Complex> {
  using type = double;
};
template<std::size_t I>
std::tuple_element_t<I, Complex>& get(Complex &c) {
  return I == 0 ? c.re : c.im;
}
int main() {
  Complex c;
  auto &[re, im] = c;
  SHOW(re, im);
}
```

```
re = 0; im = 0
```

# Behälterklassen Anmerkungen

- Elementzugriff
  - entweder über `operator []` ohne Indexprüfung → möglicherweise undefiniertes Verhalten
  - oder mit Methode `at()` → Ausnahme wird ausgelöst
- Optimierung für fortlaufende Behälter (*contiguous container*)
  - Kapazität wenn möglich vorher angeben, erspart aufwändiges Reservieren und Kopieren
- Methoden nur für Basisoperationen → Großteil der Funktionalität über Iteratoren entkoppelt und als unabhängige Algorithmen implementiert



# Iteratoren

- Verallgemeinerung von Zeiger → Positionen innerhalb Behälter
  - einheitliche Verwendung verschiedener Behälter `b`
  - Start- und End-Iterator über `b.begin()` und `b.end()`
    - alternativ, z.B. auch für C-Array `a[]` mit `std::begin(a)` und `std::end(a)`

```
std::vector<int> v = {1, 2, 3, 4, 5};
for (auto it = begin(v); it != end(v); ++it) {
  std::print("{}, ", *it);
}
PL();

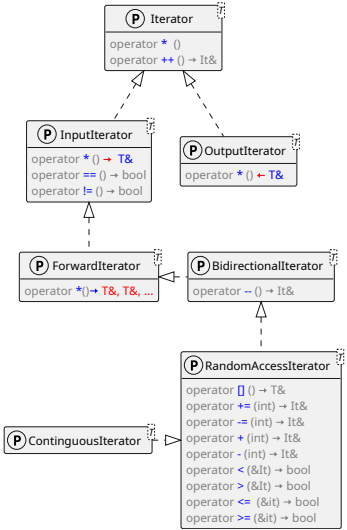
int a[] = {1, 2, 3, 4, 5};
for (auto it = std::begin(a); it != std::end(a); ++it) {
  std::print("{}, ", *it);
}
PL();

std::set<int> s = {1, 2, 3, 4, 5, 6, 7};
for (auto it = begin(s); it != end(s); ++it) {
  std::print("{}, ", *it);
}
```

```
1, 2, 3, 4, 5,
1, 2, 3, 4, 5,
1, 2, 3, 4, 5, 6, 7,
```

# Iterator-Kategorien

- Unterscheidung in verschiedene Iterator-Kategorien, je nach (effizient möglicher $O(1)$) Funktionalität in `iterator_category` (Typ-Klassen)
  - Iterator Basisfunktionalität
    - Dereferenzierung (`*`)
    - zum nächsten Element bewegen (`++`)
  - OutputIterator wie Iterator, zusätzlich
    - Element (einmalig) setzen (über `*`)
  - InputIterator wie Iterator, zusätzlich
    - Element (einmalig) lesen (über `*`)
    - Position vergleichen (`==`, `!=`)
  - ForwardIterator wie InputIterator, zusätzlich
    - Element wiederholt lesen
  - BidirectionalIterator wie ForwardIterator, zusätzlich
    - zum vorherigen Element bewegen (`--`)
  - RandomAccessIterator wie BidirectionalIterator, zusätzlich
    - Zugriff auf beliebiges Element (`[]`, `+`, `-`, `+=`, `-=`, `<`, `>`, `<=`, `>=`)
  - CongiguousIterator wie RandomAccessIterator
    - Elemente liegen nebeneinander im Speicher



# Iterator-Operationen

- Hilfsfunktionen für Iteratoren

```
std::vector<std::string> v = {"one", "two", "three"};

std::vector<std::string>::iterator it = begin(v);
SHOW(*it);

it = next(it); // NOTE std::next found via ADL
SHOW(*it);

advance(it, -1); // NOTE: std::advance
SHOW(*it);

SHOW(distance(it, v.begin()),
     distance(it, v.end()));

auto first = v.begin();
auto second = next(first);
iter_swap(first, second);
SHOW(s);
```

Iterator library: Iterator Operations - cppreference.com

# Bereichsschleifen (*range-based loops*)

- Die Bereichsschleife in C++ operiert im Hintergrund auf dazugehörigen Iteratoren und wird automatisch transformiert

```cpp
int a1[] = { 1, 2, 3 };
for (int i : a1) {
  std::print("{}, ", i);
}

std::array a2 { 1, 2, 3 };
for (int i : a2) {
  std::print("{}, ", i);
}

std::vector v { 1, 2, 3 };
for (int i : v) {
  std::print("{}, ", i);
}
```

```
1, 2, 3, 1, 2, 3, 1, 2, 3,
```

```cpp
int a1[] = { 1, 2, 3 };
for (auto it = std::begin(a1);
          it != std::end(a1); ++it) {
  int i = *it;
  std::print("{}, ", i);
}

std::array a2 { 1, 2, 3 };
for (auto it = begin(a2); it != end(a2); ++it) {
  int i = *it;
  std::print("{}, ", i);
}

std::vector v { 1, 2, 3 };
for (auto it = begin(v); it != end(v); ++it) {
  int i = *it;
  std::print("{}, ", i);
}
```

```
1, 2, 3, 1, 2, 3, 1, 2, 3,
```

# Gültigkeit von Iteratoren (i)

- Iteratoren hängen oft von dahinter liegendem Behälter ab (genauso wie Zeiger)
  - bei Veränderung des Behälters daher möglicherweise nicht mehr gültig

```
std::vector<int> v {1};
std::vector<int>::iterator it = v.begin();
SHOW(*it, v.capacity(), v.data());

v.push_back(1); // -> internal array is re-allocated
SHOW(v.capacity(), v.data());
// SHOW(*it); // XXX: undefined behaviour
```

```
*it = 1; v.capacity() = 1; v.data() = 0x7b54657e0010
v.capacity() = 2; v.data() = 0x7b54657e0330
```

```
=================================================================
==93634==ERROR: AddressSanitizer: heap-use-after-free on address 0x7b6d499e0010
READ of size 4 at 0x7b6d499e0010 thread T0
==93634==ABORTING
```

# Gültigkeit von Iteratoren (ii)

```cpp
#include <filesystem>
int main() {
    auto p = std::filesystem::current_path();
    std::println("{}", p.string());

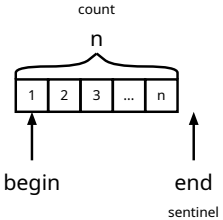
    // auto result = std::ranges::search(p.string(), "swc");

    std::string s = p.string();
    std::string swc = "swc";
    std::println("{}", s);
    auto result = std::ranges::search(s, swc);
    std::println("{} {}", result, result.empty());
}
```

```
/home/erik/workspace/swc3vl/slides
/home/erik/workspace/swc3vl/slides
['s', 'w', 'c'] false
```

# Element-Bereiche: Ranges

- Iteratoren werden oft in Paaren verwendet
- Elementbereiche (engl. *ranges*) als sicherere Alternative seit C++20
  - aus Iterator und Marker (engl. *sentinel*)
  - aus Iterator und Anzahl




- einfachere und sichere Verwendung → ein *Range*-Objekt statt z.B. zwei Iteratoren
- vermeiden Probleme mit “losen” Iteratoren
  - z.B. Bereich aufgespannt zwischen Iteratoren aus verschiedenen Behältern

Ranges library (since C++20) - cppreference.com

# Algorithmen Überblick

Umfangreiche Sammlung von Algorithmen auf Iteratoren, Ranges and Views

- Funktion auf jedes Element anwenden: `for_each`, `for_each_n`
- Prädikat prüfen: `all_of`, `any_of`, `none_of`
- Elemente suchen
  - einzelne Elemente: `contains`, `find`, `find_if`, ...
    - `binary_search`, `lower_bound`, ...
    - `min_element`, `minmax_element`, ...
  - mehrere: `search`, `count`,
- Teilsequenzen vergleichen
  - `equal`, `mismatch`
- Eigenschaften Prüfen
  - `is_sorted`, `is_partitioned`, `is_permutation`, `is_heap`, ...
- Elemente kopieren, bewegen, transformieren oder löschen
  - `copy`, `copy_if`, `copy_backward`, `move`, ...
  - `swap`, `swap_ranges`, ...
  - `transform`, `replace`, ...
  - `remove`, `remove_if`, `unique`, ... (verschiebt ungewollte Elemente ans Ende)
  - `reverse`, `rotate`, `shuffle`, ...
  - `sample`

Algorithms library - cppreference.com

# Algorithmen Beispiele mit Iteratoren

```cpp
constexpr bool is_negative(double x) { return x < 0; }
constexpr double to_farenheit(double celsius) { return celsius * 1.8 + 32; }

int main() {
  std::vector<double> temp_c = { -10, 0, 10, 20, 30 };
  SHOW(is_sorted(begin(temp_c), end(temp_c))); // NOTE: ADL call to std::is_sorted

  SHOW(temp_c);
  auto removed = remove_if(begin(temp_c), end(temp_c), is_negative);
  SHOW(temp_c); // removed elements at end
  temp_c.erase(removed, temp_c.end());
  SHOW(temp_c);

  std::vector<double> temp_f(temp_c.size());
  transform(begin(temp_c), end(temp_c), begin(temp_f), to_farenheit);
  SHOW(temp_f);
}
```

```
is_sorted(begin(temp_c) = true;
temp_c = [-10, 0, 10, 20, 30]
temp_c = [0, 10, 20, 30, 30]
temp_c = [0, 10, 20, 30]
temp_f = [32, 50, 68, 86]
```

# Algorithmen Beispiele mit Ranges

```cpp
constexpr bool is_negative(double x) { return x < 0; }
constexpr double to_farenheit(double celsius) { return celsius * 1.8 + 32; }

int main() {
  using namespace std;

  vector<double> temp_c = { -10, 0, 10, 20, 30 };
  SHOW(ranges::is_sorted(temp_c));

  SHOW(temp_c);
  auto removed = ranges::remove_if(temp_c, is_negative);
  SHOW(temp_c); // removed elements at end
  temp_c.erase(begin(removed), end(removed));
  SHOW(temp_c);

  vector<double> temp_f(temp_c.size());
  ranges::transform(temp_c, temp_f.begin(), to_farenheit);
  SHOW(temp_f);
}
```

```
ranges::is_sorted(temp_c) = true
temp_c = [-10, 0, 10, 20, 30]
temp_c = [0, 10, 20, 30, 30]
temp_c = [0, 10, 20, 30]
temp_f = [32, 50, 68, 86]
```

# Algorithmen Beispiele

```cpp
int main() {
  using namespace std;

  vector<char> v = { 'a', 'b', 'c', 'd', 'e', 'f' };
  SHOW(v);

  random_device rd;
  mt19937 rng {rd()};
  ranges::shuffle(v, rng);
  SHOW(v);
  auto it = ranges::find(v, 'a');
  println("'a' now at index {}", ranges::distance(begin(v), it));
}
```

```
v = ['a', 'b', 'c', 'd', 'e', 'f']
v = ['f', 'c', 'e', 'a', 'd', 'b']
'a' now at index 3
```

# Iterator-Adapter

- Iterator-Konzept auch relativ leicht auf anderen Kontext übertragbar
  - Ein- / Ausgabe in umgekehrter Reihenfolge → Reverse-Iterator-Adapter
  - Ausgabe mit Ausgabeoperator → Ausgabe-Iterator-Adapter
  - Element in Behälter einfügen → Einfüge-Iterator-Adapter

```
using namespace std;
vector<int> v = {1, 2, 3};
vector<int> v2(v.size());
ranges::copy(v, rbegin(v2)); // -> reverse_iterator
//              ^~~~~~ reverse-begin
SHOW(v2);

ranges::copy(v, ostream_iterator<int>(cout, ", ")); PL();

ranges::copy(v, back_inserter(v2)); // -> back_insert_iterator<vector<int>>
SHOW(v2);

set<int> s = { 3, 4, 5 };
ranges::copy(v, inserter(s, begin(s))); // -> insert_iterator<set<int>>
SHOW(s);
```

```
v2 = [3, 2, 1]
1, 2, 3,
v2 = [3, 2, 1, 1, 2, 3]
s = {1, 2, 3, 4, 5}
```

# Zeit- und Datumsfunktionen

- Template Klassen, Funktionen und Literale für Zeit und Datum in <chrono> (C++11)
  - `std::chrono::time_point<Clock, Duration>` für Repräsentation von Zeitpunkten
  - `std::chrono::duration<Rep, Period>` für Zeitspanne oder -dauer
  - `std::chrono::year_month_day` für Datum
  - Suffix `""y` für `std::chrono::year`, `""d` für Tage, Englische Monatsnamen als Konstanten
  - Operator »/« zum "*Verfeinern*" eines Datums, z.B. `2025y / December`

```cpp
using namespace std::chrono; using namespace std::chrono_literals;
time_point start = system_clock::now(); SHOW(start); // nanosecond precision

year_month_day date { floor<days>(start) };
std::println("{} = {}. {}. {}", date, date.day(), date.month(), date.year());

year_month_day xmas = date.year() / December / 24d; // compose date with literals
std::println("{} until {:%a, %d. %B '%y (CW%V)}",
             duration_cast<hours>(sys_days{xmas} - start), xmas);

std::println("calculation took {}", duration_cast<microseconds>(system_clock::now() - start));
```

```
start = 2026-09-14 13:02:55.320624111
2026-09-14 = 14. Sep. 2026
2410h until Thu, 24. December '26 (CW52)
calculation took 1932us
```

Date and time library - cppreference.com

# Beispiel Zahlen aus Datei lesen

```
std::ofstream of { "test.csv" };
of << "1, 2, 3\n4, 5, 6\n7, 8, 9\n";
of.close(); // write demo data to file

std::regex rx { "[0-9.]+" };
std::vector<std::vector<double>> table;
std::ifstream f { "test.csv" };
for (std::string line; std::getline(f, line);) { // parse line-by-line
  std::vector<double> row;
  auto it = std::sregex_iterator(line.begin(), line.end(), rx);
  for (; it != std::sregex_iterator(); ++it) {
    std::istringstream iss { it->str() };
    double d;
    iss >> d;
    row.push_back(d);
  }
  table.push_back(row);
}
std::println("{}", table);
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

# 14.3 C++ Funktoren, Lambda-Ausdrücke & Views

# Wiederholung: Funktionszeiger

- jeder Funktionsname seht für die Adresse einer Funktion im Code-Segment
- kann, wie andere Zeiger als Parameter übergeben werden

```
template <typename T>
using visitor = void (*)(T);

void print(auto i) { std::print("{}, ", i); }

int main() {
  std::vector v {1, 2, 3};
  visitor<int> vis = print<int>;
  std::ranges::for_each(v, vis);
  std::ranges::for_each(v, print<int>);
};
```

```
1, 2, 3, 1, 2, 3,
```

- kann aber keinen “Zustand” halten (außer durch globale Variablen)
- Erweiterung mit Hilfe von `std::function`, erlaubt ebenfalls Funktionszeiger

```
template <typename T>
using visitor = std::function<void(T)>;

void print(auto i) { std::print("{}, ", i); }

int main() {
  std::vector v {1, 2, 3};
  visitor<int> vis = print<int>;
  std::ranges::for_each(v, vis);
  std::ranges::for_each(v, print<int>);
};
```

```
1, 2, 3, 1, 2, 3,
```

# Funktoren

```
struct square {

  int & counter;

  square(int &c) : counter{c} {}

  int operator () (int x) {
    counter++;
    return x * x;
  }

};

int main() {
  int c = 0;
  square sqr {c};
  SHOW(sqr(2), c);

  std::vector v = { 1, 2, 3 };
  std::ranges::transform(v, v.begin(), sqr);
  SHOW(v, c);
}
```

Objekte mit Funktionsaufrufoperator
»`operator () (...)`«

- Verhalten wie Funktionen
- können Zustand oder Referenzen haben

```
sqr(2) = 4; c = 0
v = [1, 4, 9]; c = 4
```

- komfortabel in Kombination mit Algorithmen der Standardbibliothek

# Anonyme Funktoren (“*lambda expressions*”)

- kompakter Syntax um “*Quelltextschnipsel*” in Funktoren zu verpacken
  - Syntax `[ /* captures */ ]( /*parameters*/ ){ /* implementation */ }` bestehend aus
  - Liste von *“eingefangenen”* Variablen (*captures*) in Eckigen Klammern »`[]`«
    - per Kopie, z.B. »`[a, b, c]`«, oder »`[=]`« → automatisch
    - per Referenz, z.B »`[&a, &b, &c]`«, oder »`[&]`« → automatisch
  - Parameter-Liste in Runden Klammern »`()`« wie normale Funktion oder Funktionsaufrufoperator
  - Implementierung des Methodenrumpfes in geschwungenen Klammern »`{}`«

```
auto lambda = [](int i) {
  return i * i;
};
std::vector v {1, 2, 3};
std::ranges::transform(v, begin(v), lambda);
SHOW(v);
```

```
v = [1, 4, 9]
```

```
auto empty_lambda = [](){};
empty_lambda();
[](){}();
```

```
int count = 0;
auto lambda = [&count](int i) {
  count++;
  return i * i;
};
std::vector v {1, 2, 3};
std::ranges::transform(v, begin(v), lambda);
SHOW(v, count);
```

```
v = [1, 4, 9]; count = 3
```

# Views (i)

- Algorithmen teilweise auch als Views verfügbar
- werden erst bei Abfrage der Werte ausgeführt → lazy evaluation
- gute Optimierbarkeit durch den Compiler und hohe Lesbarkeit
  - Vorsicht bei “Wiederverwendung” insbesondere von Teilen der “Pipeline”

- ohne Views: umständliches Entfernen

```
std::vector v {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

auto removed = std::ranges::remove_if(v,
  [](auto i){
    return i % 2 != 0;
  });

v.erase(begin(removed), end(removed));

SHOW(v);
```

```
v = [2, 4, 6, 8, 10]
```

- algorithmische Transformation entkoppelt von Behälter-Änderung

- mit Views: direkte Transformation

```
std::vector v {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

auto evens = std::views::filter(v,
  [](auto i) {
    return i % 2 == 0;
  });

auto odds = std::views::filter(v,
  [](auto i) {
    return i % 2 != 0;
  });

SHOW(evens, odds);
```

```
evens = [2, 4, 6, 8, 10]; odds = [1, 3, 5, 7, 9]
```

# Views (ii)

- Views erlauben Verkettung durch »`operator | ()`«

```
using namespace std::views;
std::vector v {1, 2, 3, 4, 5, 6};

auto is_even = [](auto x) { return x % 2 == 0; };
auto increment = [](auto x) { return x + 1; };
auto square = [](auto x) { return x * x; };

// NOTE nested calls: harder to read
auto r1 = transform(transform(filter(v, is_even), increment), square);

// NOTE: linear transformation chains
auto r2 = v
  | filter(is_even)       // -> filter_view<vector>
  | transform(increment)  // -> transform_view<filter_view<vector>>
  | transform(square);    // -> transform_view<transform_view<filter_view<vector>>>

SHOW(r1, r2);
```

```
r1 = [9, 25, 49]; r2 = [9, 25, 49]
```

siehe RangeAdaptorClosureObject und std::ranges::range_adaptor_closure

# Views Beispiele

- Werte ohne Behälter iterieren

```
auto v = std::views::iota(1, 11); // 1, ..., 10
auto evens = v | std::views::filter(
  [](auto i) {
    return i % 2 == 0;
  });
SHOW(evens);
```

```
evens = [2, 4, 6, 8, 10]
```

- Filterkette mit unendlicher Sequenz

```
auto v = std::views::iota(1); // 1, 2, ...
auto evens = v
  | std::views::filter(
    [](auto i) {
      return i % 2 == 0;
    })
  | std::views::take(10);
SHOW(evens);
```

```
evens = [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

```
constexpr auto is_prime = [](auto i) {
  return i <= 2 ||
    std::ranges::all_of(
      std::views::iota(2, i-1),
      [&](auto j) {
        return i % j != 0;
      });
};

auto numbers = std::views::iota(1);

PL("{}", numbers | std::views::take(5));

PL("{}", numbers
   | std::views::transform(is_prime)
   | std::views::take(5));

PL("{}", numbers
   | std::views::filter(is_prime)
   | std::views::drop(100)
   | std::views::take(10));
```

```
[1, 2, 3, 4, 5]
[true, true, true, false, true]
[541, 547, 557, 563, 569, 571, 577, 587, 593, 599]
```

# Beispiel: Sum of Even Squares

```
using namespace std;
// vector<int> numbers {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
auto evens =
  views::iota(1)
| views::filter([](auto x) { return x % 2 == 0;})
| views::transform([](auto x) { return x * x; })
| views::take(3);
// | ranges::to<vector<int>>();
// auto s = ranges::fold_left(evens, 0, [](int a, int b) { return a + b; });
auto s = ranges::fold_left(evens, 0, std::plus{});
std::println("{} = {}", evens, s);
```

```
[4, 16, 36] = 56
```